KEY TERM

Linear search: checking each element of an array in turn for a required value

TASK 13.04

Use the algorithm in Worked Example 13.02 as a design pattern. Write an algorithm using the arrays from Task 13.03 to search for a friend's name and output their age.

WORKED EXAMPLE 13.03

Sorting elements in a 1D array

The simplest way to sort an unordered list of values is the following method.

1. Compare the first and second values. If the first value is larger than the second value, swap them.
2. Compare the second and third values. If the second value is larger than the third value, swap them.
3. Compare the third and fourth values. If the third value is larger than the fourth value, swap them.
4. Keep on comparing adjacent values, swapping them if necessary, until the last two values in the list have been processed.

Figure 13.03 shows what happens to the values as we work down the array, following this algorithm.

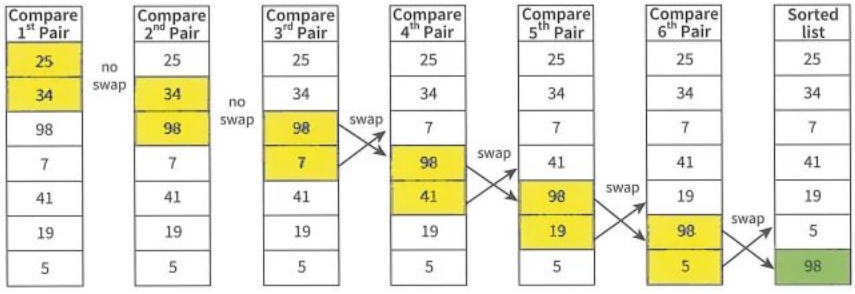


Figure 13.03 Swapping values working down the array

When we have completed the first pass through the entire array, the largest value is in the correct position at the end of the array. The other values may or may not be in the correct order.

We need to work through the array again and again. After each pass through the array the next largest value will be in its correct position, as shown in Figure 13.04.

(Continued)

Original list	After pass 1	After pass 2	After pass 3	After pass 4	After pass 5	After pass 6
25	25	25	7	7	7	5
34	34	7	25	19	5	7
98	7	34	19	5	19	19
7	41	19	5	25	25	25
41	19	5	34	34	34	34
19	5	41	41	41	41	41
5	98	98	98	98	98	98

Figure 13.04 States of the array after each pass

In effect we perform a loop within a loop, a nested loop. This method is known as a **bubble sort**. The name comes from the fact that smaller values slowly rise to the top, like bubbles in a liquid.

The identifiers needed for the algorithm are listed in Table 13.03.

Identifier	Data type	Explanation
MyList	ARRAY[0:6] OF INTEGER	Data structure (1D array) to store seven numbers
MaxIndex	INTEGER	The upper bound of the array
n	INTEGER	The number of pairs of elements to compare in each pass
i	INTEGER	Counter for outer loop
j	INTEGER	Counter for inner loop
Temp	INTEGER	Variable for temporary storage while swapping values

Table 13.03 Identifier table for bubble sort algorithm

The algorithm in pseudocode is:

```
n ← MaxIndex - 1
FOR i ← 0 TO MaxIndex - 1
    FOR j ← 0 TO n
        IF MyList[j] > MyList[j + 1]
          THEN
            Temp ← MyList[j]
            MyList[j] ← MyList[j + 1]
            MyList[j + 1] ← Temp
        ENDIF
    NEXT j
    n ← n - 1 // this means the next time round the inner loop, we don't
              // look at the values already in the correct positions.
NEXT i
```

(Continued)

The values to be sorted may already be in the correct order before the outer loop has been through all its iterations. Look at the list of values in Figure 13.05. It is only slightly different from the first list we sorted.

Original list	After pass 1	After pass 2	After pass 3	After pass 4	After pass 5	After pass 6
5	5	5	5	5	5	5
34	34	7	7	7	7	7
98	7	34	19	19	19	19
7	41	19	25	25	25	25
41	19	25	34	34	34	34
19	25	41	41	41	41	41
25	98	98	98	98	98	98

Figure 13.05 States of the list after each pass

After the third pass the values are all in the correct order but our algorithm will carry on with three further passes through the array. This means we are making comparisons when no further comparisons need to be made.

If we have gone through the whole of the inner loop (one pass) without swapping any values, we know that the array elements must be in the correct order. We can therefore replace the outer loop with a conditional loop.

We can use a variable `NoMoreSwaps` to store whether or not a swap has taken place during the current pass. We initialise the variable `NoMoreSwaps` to `TRUE`. When we swap a pair of values we set `NoMoreSwaps` to `FALSE`. At the end of the pass through the array we can check whether a swap has taken place.

The identifier table for this improved algorithm is shown in Table 13.04.

Identifier	Data type	Explanation
`MyList`	`ARRAY[0:6] OF INTEGER`	Data structure (1D array) to store seven numbers
`MaxIndex`	`INTEGER`	The upper bound of the array
`n`	`INTEGER`	The number of pairs of elements to compare in each pass
`NoMoreSwaps`	`BOOLEAN`	`TRUE` when no swaps have occurred in current pass `FALSE` when a swap has occurred
`j`	`INTEGER`	Counter for inner loop
`Temp`	`INTEGER`	Variable for temporary storage while swapping values

Table 13.04 Identifier table for improved bubble sort algorithm

This improved algorithm in pseudocode is:

```
n ← MaxIndex - 1
REPEAT
    NoMoreSwaps ← TRUE
    FOR j ← 0 TO n
        IF MyList[j] > MyList[j + 1]
```

(Continued)

```
            THEN
              Temp ← MyList[j]
              MyList[j] ← MyList[j + 1]
              MyList[j + 1] ← Temp
              NoMoreSwaps ← FALSE
          ENDIF
      NEXT j
      n ← n - 1
UNTIL NoMoreSwaps = TRUE
```

KEY TERM

Bubble sort: a sort method where adjacent pairs of values are compared and swapped

Discussion Point:

What happens if the array elements are already in the correct order?

TASK 13.05

Rewrite the algorithm in Worked Example 13.03 to sort the array elements into descending order.

13.05 Two-dimensional arrays

When we write a table of data (a matrix) on a piece of paper and want to refer to individual elements of the table, the convention is to give the row number first and then the column number. When declaring a 2D array, the number of rows is given first, then the number of columns. Again we have lower and upper bounds for each dimension.

In pseudocode, a 2D array declaration is written as:

```
DECLARE <identifier> : ARRAY[<lBound1>:<uBound1>,
<lBound2>:<uBound2>] OF <dataType>
```

The array elements in a 2D array can be numbered from 0. Sometimes it is more intuitive to use rows from row 1 and columns from column 1, as shown with the board game in Worked Example 13.04 and 13.05.

To declare a 2D array to represent a game board of six rows and seven columns, the pseudocode statement is:

```
Board : ARRAY[1:6,1:7] OF INTEGER
```

Accessing 2D arrays

A specific element in a table is accessed using an index pair. In pseudocode this is written as:

```
<arrayIdentifier>[x,y]
```

Pseudocode example:

```
Board[3,4] ← 0 // sets the element in row 3 and column 4 to zero
```